SLOT 6

UI BASICS – LIST & INTERACTION

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- UI Basics – Scaffold Layout
- ListView & Card/ListTile
- Interactive Widgets

Giúp sinh viên **nhìn một màn hình app → hiểu cấu trúc widget → tự xây dựng lại UI + tương tác cơ bản**, làm nền tảng cho các màn hình danh sách và thao tác người dùng trong ứng dụng Flutter thực tế.

1. UI BASICS – Scaffold & Cấu trúc màn hình

- **Khái niệm cốt lõi cần hiểu**
  - `Scaffold` là **khung xương của 1 màn hình app**
  - Mỗi màn hình Flutter thường có: `AppBar`, `Body`, `FloatingActionButton`
  - UI Flutter được xây từ **widget lồng widget**
- **Mục tiêu học tập**
  - Tạo được **một màn hình hoàn chỉnh** với tiêu đề, nội dung chính
  - Biết đặt widget đúng vị trí trong `Scaffold`
- **Ý nghĩa sư phạm / thực tiễn**
  - Scaffold giúp sinh viên **định hình tư duy "1 màn hình = 1 cấu trúc UI rõ ràng"**
  - Là nền tảng cho **mọi màn hình app thật** (Home, List, Detail, Profile…)

2. ListView – Hiển thị danh sách dữ liệu

- **Khái niệm cốt lõi cần hiểu**
  - `ListView` dùng để hiển thị **nhiều phần tử giống nhau theo chiều dọc**
  - Scroll tự động khi dữ liệu dài
  - Phân biệt `ListView`, `Column`, `SingleChildScrollView` (mức cơ bản)
- **Mục tiêu học tập**
  - Hiển thị được **danh sách dữ liệu đơn giản**
  - Biết dùng `ListView.builder` cho danh sách nhiều phần tử
- **Ý nghĩa sư phạm / thực tiễn**
  - **90% app thực tế đều có danh sách**: sản phẩm, tin tức, người dùng, lịch sử
  - Sinh viên bắt đầu tư duy: **UI lặp → dữ liệu → item**

3. Card & ListTile – Chuẩn hóa item trong danh sách

- **Khái niệm cốt lõi cần hiểu**
  - `Card` là khung UI giúp item **rõ ràng – dễ nhìn**
  - `ListTile` là widget **tối ưu cho 1 dòng thông tin** (icon – title – subtitle)
- **Mục tiêu học tập**
  - Thiết kế được **item danh sách đẹp – gọn – dễ đọc**
  - Kết hợp `Card + ListTile` để tạo danh sách giống app thật
- **Ý nghĩa sư phạm / thực tiễn**
  - Giúp sinh viên **xây UI giống sản phẩm thương mại**
  - Tránh việc tự layout phức tạp khi chưa vững widget cơ bản

4. Interactive Widgets – Button & sự kiện người dùng

- **Khái niệm cốt lõi cần hiểu**
  - Button là widget **nhận hành động từ người dùng**
  - `onPressed` là nơi xử lý sự kiện
  - Button không chỉ là UI mà là **điểm bắt đầu logic**

PhucPT10@fpt.edu.vn

- **Mục tiêu học tập**
  - Tạo được các loại nút cơ bản (`ElevatedButton`, `TextButton`)
  - Xử lý được sự kiện click đơn giản (print, đổi UI, điều hướng giả lập)
- **Ý nghĩa sư phạm / thực tiễn**
  - Chuyển từ **UI tĩnh → UI có phản hồi**
  - Mở đầu cho tư duy **UI + logic = app hoạt động**

5. Gesture & onTap – Tương tác trên List Item

- **Khái niệm cốt lõi cần hiểu**
  - Không chỉ button mới click được
  - `GestureDetector` / `InkWell` giúp **bắt sự kiện tap trên bất kỳ widget nào**
  - `onTap` dùng nhiều trong danh sách
- **Mục tiêu học tập**
  - Làm cho **item trong ListView có thể bấm được**
  - Xử lý hành vi người dùng khi chạm vào từng item
- **Ý nghĩa sư phạm / thực tiễn**
  - Đây là cách app thực tế hoạt động:
    - Bấm item → xem chi tiết
    - Bấm dòng → mở màn hình khác
  - Sinh viên bắt đầu tư duy **UI phản ứng theo hành vi người dùng**

6. Kết nối UI – List – Interaction thành một màn hình hoàn chỉnh

- **Khái niệm cốt lõi cần hiểu**
  - UI không đứng riêng lẻ
  - Màn hình app = Scaffold + List + Item + Interaction
- **Mục tiêu học tập**
  - Tự xây dựng **1 màn hình danh sách có thể tương tác**
  - Hiểu luồng: **hiển thị → cuộn → bấm → phản hồi**
- **Ý nghĩa sư phạm / thực tiễn**
  - Đây là **bước chuyển từ học widget → làm app**
  - Chuẩn bị cho các slot sau: state, navigation, data động

TÀI LIỆU THAM KHẢO (ƯU TIÊN NGƯỜI MỚI)

1. **Flutter Official Docs – UI & Widgets**
   https://docs.flutter.dev/ui
   → Chuẩn chính thống, hình ảnh rõ, dễ tra cứu
2. **Flutter for Beginners – Alessandro Biessek**
   → Sách giải thích chậm, nhiều ví dụ UI thực tế
3. **Flutter UI Basics – Flutter YouTube Official**
   → Video ngắn, trực quan, phù hợp sinh viên yếu
4. **Academind – Flutter UI Crash Course**
   → Giải thích dễ hiểu, liên hệ app thật
5. **Pragmatic Flutter – UI & Layout Section**
   → Dành cho người muốn hiểu bản chất widget
   -

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

UI BASICS – LIST & INTERACTION

Tư tưởng xuyên suốt Slot 6:

1 màn hình app = 1 cấu trúc UI rõ ràng + danh sách dữ liệu + hành vi người dùng

1. Scaffold Layout & UI Structure (NỀN TẢNG BẮT BUỘC)

PHẢI HIỂU

- `Scaffold` là **khung giao diện tiêu chuẩn** cho **1 screen Flutter**
- Mỗi screen thường có **1 Scaffold duy nhất**
- `AppBar`, `body`, `FloatingActionButton` là **các vùng chức năng**, không phải widget rời rạc
- `MaterialApp` ≠ `Scaffold`
  - `MaterialApp` → cấu hình toàn app
  - `Scaffold` → layout cho từng màn hình

PHẢI LÀM ĐƯỢC

- Tạo 1 screen hoàn chỉnh với:
  - AppBar có title
  - Body hiển thị nội dung
- Biết khi nào:
  - **Cần Scaffold** (màn hình chính)
  - **Không cần Scaffold** (widget con)

PHẢI TRÁNH LỖI

- Quên Scaffold → UI hiển thị sai Material Design
- Nhét quá nhiều widget & logic trực tiếp vào `body`
- Dùng nhiều Scaffold lồng nhau sai mục đích
- Nhầm vai trò `MaterialApp` và `Scaffold`

BEST PRACTICES

- **1 screen = 1 Scaffold**
- Scaffold chỉ lo **bố cục**, không xử lý logic
- Giữ Scaffold:
  - gọn
  - dễ đọc
  - dễ mở rộng

Ý nghĩa sư phạm:

Giúp sinh viên **xây UI có cấu trúc chuẩn**, tránh kiểu "vẽ được nhưng không phải app".

2. ListView – Hiển thị danh sách dữ liệu (TRÁI TIM CỦA APP)

PHẢI HIỂU

- `ListView` dùng để hiển thị **nhiều item giống nhau**
- `ListView.builder` sinh item **khi cần** → hiệu quả cho dữ liệu động
- `itemBuilder` = cách vẽ từng item
- `itemCount` = số lượng item

PHẢI LÀM ĐƯỢC

- Dùng `ListView.builder` đúng cách
- Gắn dữ liệu từ `List` vào UI
- Hiển thị danh sách có thể scroll mượt

PHẢI TRÁNH LỖI

- Dùng `Column` cho danh sách dài → **overflow**
- Quên `itemCount` → lỗi logic
- Nhét `ListView` trong `Column` sai cách
- Viết UI + logic lẫn lộn trong `itemBuilder`

PhucPT10@fpt.edu.vn

✅ BEST PRACTICES

- Dữ liệu nhiều → **luôn dùng ListView.builder**
- Tách **UI item** thành widget riêng
- `itemBuilder` chỉ lo **render UI**

🧠 **Ý nghĩa sư phạm:**
Sinh viên bắt đầu hiểu:
App = dữ liệu → lặp → hiển thị → scroll

3. Card & ListTile – Chuẩn hóa UI danh sách

📌 PHẢI HIỂU

- `Card` = khung giúp **phân tách nội dung**
- `ListTile` = layout **chuẩn cho 1 item danh sách**
- `ListTile` đã giải quyết sẵn:
  - icon
  - text
  - khoảng cách

🛠 PHẢI LÀM ĐƯỢC

- Sử dụng `ListTile` đúng cấu trúc:
  - `leading`
  - `title`
  - `subtitle`
  - `trailing`
- Bọc `ListTile` trong `Card`
- Tùy chỉnh spacing cơ bản

❌ PHẢI TRÁNH LỖI

- Dùng `Container` phức tạp thay `ListTile`
- Hardcode layout từng item
- UI rối, không đồng nhất giữa các item

✅ BEST PRACTICES

- Danh sách chuẩn → **ưu tiên ListTile**
- Card dùng để **nhóm & phân tách thông tin**
- UI đơn giản > UI cầu kỳ

🧠 **Ý nghĩa sư phạm:**
Giúp sinh viên **code nhanh – đúng – giống app thật**, không "vẽ tay" UI phức tạp.

4. Interactive Widgets – Tương tác người dùng (UI CÓ HÀNH VI)

📌 PHẢI HIỂU

- UI không chỉ hiển thị → **phải phản hồi**
- `onPressed`, `onTap` là **cầu nối UI ↔ hành động**
- Không phải widget nào cũng có tương tác sẵn

🛠 PHẢI LÀM ĐƯỢC

- Dùng thành thạo:
  - `ElevatedButton`
  - `IconButton`
  - `GestureDetector`
- Bắt sự kiện tap / click
- Gọi hàm xử lý khi người dùng tương tác

❌ PHẢI TRÁNH LỖI

- Quên callback → nút "chết"
- Viết logic dài trong `onTap`
- Nhầm vai trò Button và GestureDetector

PhucPT10@fpt.edu.vn

✅ BEST PRACTICES

- UI **chỉ gọi hàm**
- Logic để riêng (function / class khác)
- Chuẩn bị tư duy reactive sau này

🧠 **Ý nghĩa sư phạm:**
Đây là bước chuyển:
UI tĩnh → app có hành vi

🔷 5. Kết hợp List + Interaction (TƯ DUY APP THỰC TẾ)

📌 PHẢI HIỂU

- Mỗi item trong list **đều có thể tương tác**
- Interaction **không làm vỡ layout**
- UI cần biết **item nào đang được click**

🛠 PHẢI LÀM ĐƯỢC

- Gắn `onTap` cho `ListTile`
- Xác định item được click thông qua index / data
- Debug hành vi UI khi người dùng tương tác

❌ PHẢI TRÁNH LỖI

- Không biết item nào được click
- Logic rải rác khắp UI
- Rebuild UI không cần thiết

✅ BEST PRACTICES

- Truyền dữ liệu rõ ràng vào item
- Tách widget item riêng
- Chuẩn bị cho:
  - state
  - navigation
  - reactive pattern

🧠 **Ý nghĩa sư phạm:**
Giúp sinh viên **bước vào tư duy app thật**, không chỉ là bài tập UI.

🎯 MỤC TIÊU CUỐI SLOT 6

Sau Slot 6, sinh viên **BẮT BUỘC** phải:

- Hiểu:
  - UI Flutter được xây dựng theo **widget tree**
  - Vai trò từng widget trong màn hình
- Làm được:
  - ✅ 1 màn hình hoàn chỉnh (Scaffold chuẩn)
  - ✅ 1 danh sách dữ liệu có scroll
  - ✅ Item trong danh sách có tương tác
- Phân biệt được:
  - **UI đúng tư duy Flutter**
  - ❌ UI "chạy được nhưng sai kiến trúc"

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

UI BASICS – LIST & INTERACTION

Thông điệp cốt lõi của Slot 6:

UI không phải để "cho đẹp" → UI là cách người dùng tương tác với app

1 ỨNG DỤNG SLOT 6 TRONG PHÁT TRIỂN DỰ ÁN FLUTTER THỰC TẾ

A. Xây dựng màn hình UI hoàn chỉnh (Screen-level Thinking)

Ứng dụng thực tế

- Mỗi màn hình trong app thật:
  - Home
  - List
  - Detail
  - Profile

    → **đều được tổ chức quanh 1 `Scaffold`**

Vai trò kỹ thuật Slot 6

- `Scaffold`:
  - chia rõ AppBar – body – action
  - giúp UI nhất quán giữa các screen
- `StatelessWidget / StatefulWidget`:
  - Stateless: UI hiển thị đơn thuần
  - Stateful: UI có thay đổi khi tương tác

👉 **Ý nghĩa thực tế**

Giúp team:

- dễ chia task theo screen
- dễ maintain UI khi app lớn dần

B. Hiển thị danh sách dữ liệu (Data → UI)

Ứng dụng thực tế

- Hầu hết app đều xoay quanh **danh sách**:
  - sản phẩm
  - tin tức
  - công việc
  - sinh viên
  - nhân viên

Vai trò kỹ thuật Slot 6

- `ListView.builder`:
  - render item theo dữ liệu
  - scroll mượt, không lag
- `Card + ListTile`:
  - chuẩn hóa UI item
  - nhìn giống app thương mại

👉 **Ý nghĩa thực tế**

- Build MVP **rất nhanh**
- UI danh sách **đồng nhất – dễ mở rộng**

C. Xử lý tương tác người dùng (UI có hành vi)

Ứng dụng thực tế

- Người dùng:
  - bấm item
  - bấm nút
  - chọn hành động

Vai trò kỹ thuật Slot 6

PhucPT10@fpt.edu.vn

- `ElevatedButton`, `IconButton` → hành động rõ ràng
- `GestureDetector`, `onTap` → tương tác trên item
- `setState` → cập nhật UI khi có thay đổi nhỏ

👉 **Ý nghĩa thực tế**
UI **bắt đầu "sống"**, không còn là màn hình tĩnh.

2 CÁC TÌNH HUỐNG THỰC TẾ (REAL-WORLD SCENARIOS)

◈ TÌNH HUỐNG 1: APP QUẢN LÝ CÔNG VIỆC CÁ NHÂN (TODO APP)

Bối cảnh
- App nhỏ cho cá nhân
- Hiển thị danh sách công việc
- Click để đánh dấu hoàn thành

Vấn đề ban đầu
- Dùng `Column` hiển thị danh sách → overflow
- UI rối, khó đọc
- Click item không rõ phản hồi

Áp dụng Slot 6
- `Scaffold`:
  - AppBar: "My Tasks"
  - body: ListView
- `ListView.builder`:
  - render danh sách task
- `Card + ListTile`:
  - title: tên task
  - trailing: icon trạng thái
- `onTap + setState`:
  - click → đổi trạng thái task

Mục tiêu đạt được
- UI rõ ràng
- Code dễ hiểu
- Người dùng thao tác tự nhiên

◈ TÌNH HUỐNG 2: APP SINH VIÊN – DANH SÁCH MÔN HỌC

Bối cảnh
- App nội bộ cho sinh viên
- Hiển thị danh sách môn học
- Click để xem chi tiết

Vấn đề ban đầu
- Hardcode UI từng môn
- Mỗi item layout khác nhau
- Code UI khó bảo trì

Áp dụng Slot 6
- `Scaffold` cho screen "Subjects"
- `ListView.builder` gắn với `List<Subject>`
- `Card + ListTile`:
  - leading: icon môn
  - title: tên môn
  - subtitle: số tín chỉ
- `onTap`:
  - log môn được chọn (chuẩn bị cho navigation)

PhucPT10@fpt.edu.vn

Mục tiêu đạt được

- Chuẩn hóa UI
- Thêm / xóa môn học dễ dàng
- Chuẩn bị tốt cho screen chi tiết

TÌNH HUỐNG 3: MVP STARTUP – DANH SÁCH SẢN PHẨM

Bối cảnh

- MVP cần demo nhanh
- Danh sách sản phẩm cơ bản
- Có nút "Add to cart"

Vấn đề ban đầu

- UI làm nhanh nhưng thiếu nhất quán
- Click không rõ tác dụng
- Code khó chỉnh sửa khi đổi UI

Áp dụng Slot 6

- `Scaffold` chia rõ layout
- `ListView.builder` hiển thị sản phẩm
- `Card` giúp item nổi bật
- `ListTile + IconButton`:
  - icon thêm vào giỏ
- `setState`:
  - cập nhật UI số lượng

Mục tiêu đạt được

- Build MVP nhanh
- Demo mượt
- Dễ nâng cấp lên state management sau

3 CASE STUDY – APP SINH VIÊN (MÔ PHỎNG)

Bối cảnh dự án

- App quản lý học tập cho sinh viên
- Màn hình chính: danh sách môn học

Vấn đề ban đầu

- UI chắp vá
- Danh sách dài → lỗi hiển thị
- Code UI + logic trộn lẫn

Áp dụng Slot 6

- `Scaffold`:
  - chuẩn hóa screen
- `ListView.builder`:
  - hiển thị môn học
- `Card + ListTile`:
  - layout gọn, đồng nhất
- `onTap`:
  - chọn môn học
- `setState`:
  - highlight item đang chọn

Kết quả đạt được

- UI rõ ràng, dễ nhìn
- Code dễ đọc, dễ dạy lại
- Sinh viên thao tác không cần hướng dẫn nhiều

PhucPT10@fpt.edu.vn

4 BÀI HỌC RÚT RA (LESSONS LEARNED)

🎯 ĐIỀU QUAN TRỌNG NHẤT KHI XÂY UI FLUTTER

- **Tư duy theo screen, không theo widget rời**
- UI phải:
  - rõ
  - nhất quán
  - có hành vi

⚠️ SAI LẦM PHỔ BIẾN SINH VIÊN

- Lạm dụng `Column` cho danh sách
- Viết logic xử lý trực tiếp trong UI
- Không tách widget item
- UI "chạy được" nhưng khó mở rộng

✅ NGUYÊN TẮC DUY TRÌ LÂU DÀI

- 1 screen = 1 Scaffold
- Danh sách → ListView.builder
- UI item → Card / ListTile
- Interaction → rõ ràng, dễ hiểu
- `setState` chỉ dùng cho UI đơn giản

🧷 KẾT LUẬN

Slot 6 là **bước ngoặt quan trọng**:

- ❌ Trước: *"Em vẽ được UI rồi"*
- ✅ Sau: *"Em xây được 1 màn hình app có hành vi"*

👉 **Liên kết cốt lõi sinh viên cần ghi nhớ:**
UI đúng → UX tốt → App dễ mở rộng → Dự án làm được thật

PhucPT10@fpt.edu.vn

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

UI BASICS – LISTVIEW – INTERACTIVE WIDGETS

Tư tưởng xuyên suốt:

Flutter không phải "vẽ UI" → Flutter là "mô tả giao diện + hành vi theo Screen"

A PHẦN A. UI BASICS – SCREEN & LAYOUT

(Nền tảng tư duy UI Flutter)

I. TƯ DUY NỀN TẢNG UI TRONG FLUTTER

◈ Screen trong Flutter là gì?

- **Screen** = một màn hình mà người dùng nhìn thấy tại một thời điểm
- Ví dụ:
  - Home Screen
  - Product List Screen
  - Detail Screen

👉 Trong Flutter:

- Screen **không phải file**
- Screen là **một widget lớn**, thường là `StatelessWidget` hoặc `StatefulWidget`

◈ UI ≠ Layout rời rạc

- Flutter **không khuyến khích**:
  - nhét toàn bộ UI vào 1 file dài
  - ghép widget rời rạc không có cấu trúc

👉 Flutter hướng tới:

- UI có **cấu trúc rõ ràng**
- Dễ hiểu:
  - screen nào
  - widget nào là con

◈ Tư duy cốt lõi: 1 màn hình = 1 Scaffold

- Mỗi screen **độc lập**
- Có:
  - AppBar
  - Body
  - Action riêng

✕ Sinh viên thường hiểu sai

- Dồn nhiều màn hình vào 1 widget
- Không phân biệt:
  - screen
  - widget con (item, button, card…)

🧿 Liên hệ hệ điều hành khác

- Android → `Activity`
- iOS → `ViewController`
- Web → `Page`

🧠 **Ý nghĩa học thuật:**

Flutter thiết kế UI theo **Screen-based architecture** → dễ mở rộng app nhiều màn hình.

PhucPT10@fpt.edu.vn

II. SCAFFOLD – KHUNG SCREEN UI

◈ Scaffold là gì?

- `Scaffold` là **bộ khung tiêu chuẩn** cho 1 screen Flutter
- Cung cấp sẵn:
  - AppBar
  - Body
  - FloatingActionButton
  - Drawer, BottomBar (sau này)

◈ Vai trò của Scaffold

- Tách rõ:
  - **khung giao diện**
  - **nội dung hiển thị**

👉 Scaffold **không lo logic**, chỉ lo bố cục.

◈ Mối quan hệ tổng thể

```
MaterialApp
  └── Scaffold
        ├── AppBar
        └── Body (UI chính)
```

🏠 Ví dụ minh họa

- **Home Screen**
  - AppBar: "Home"
  - Body: danh sách
- **List Screen**
  - AppBar: "Products"
  - Body: ListView

✕ Nếu không dùng Scaffold

- AppBar hiển thị sai
- Material style không hoạt động
- UI "trông giống Flutter demo lỗi"

🧠 **Ẩn dụ sư phạm:**
Scaffold giống **bộ khung nhà**
→ không có khung thì nội thất không đứng vững.

III. BODY & NGUYÊN LÝ "ONE ROOT WIDGET"

◈ Body chỉ nhận 1 widget

- Đây là quy tắc **bắt buộc** trong Flutter
- Widget đó gọi là **root widget**

❓ Vì sao Flutter thiết kế như vậy?

- UI Flutter là **Widget Tree**
- Mỗi node chỉ có **1 cha**
- Giữ cấu trúc rõ ràng, dễ render

◈ Cách xử lý khi có nhiều widget

- Dùng widget chứa:
  - `Column` → dọc
  - `Row` → ngang

PhucPT10@fpt.edu.vn

- `Center` → căn giữa

⚠ Lỗi sinh viên hay gặp

- Dùng `Column` bừa bãi
- Không hiểu overflow
- Nhét ListView trong Column sai cách

🧠 **Ý nghĩa học thuật:**

Flutter ưu tiên **cấu trúc cây rõ ràng** hơn là "vẽ tự do".

IV. LISTVIEW – DANH SÁCH TRONG UI THỰC TẾ

◈ Vì sao Column không phù hợp cho danh sách dài?

- Column:
  - render **tất cả widget cùng lúc**
  - không tối ưu scroll
- Dữ liệu dài → lag, overflow

◈ ListView là gì?

- Widget hiển thị **danh sách cuộn**
- Chỉ render phần đang nhìn thấy

◈ ListView vs ListView.builder

	ListView	ListView.builder
Dữ liệu	Ít	Nhiều / động
Hiệu năng	Trung bình	Tốt
App thật	❌	✅

◈ Phân tích hiệu năng

- `ListView.builder`:
  - sinh item **khi cần**
  - tiết kiệm bộ nhớ
  - bắt buộc cho app thật

🧠 **Ý nghĩa học thuật:**

Danh sách = **Data-driven UI**, không phải UI cứng.

V. CARD & LISTTILE – CHUẨN HÓA UI DANH SÁCH

◈ Card dùng để làm gì?

- Tạo khối thông tin riêng biệt
- Giúp UI dễ đọc, phân tách rõ

◈ ListTile giải quyết vấn đề gì?

- Layout chuẩn cho:
  - icon
  - text
  - action
- Không cần tự ghép `Row + Padding`

◈ Vì sao nên dùng ListTile?

- Nhất quán UI
- Code ngắn
- Ít lỗi layout

PhucPT10@fpt.edu.vn

🧠 **Ý nghĩa UX:**
Người dùng quen cách đọc danh sách → ListTile đúng chuẩn này.

VI. INTERACTIVE WIDGETS – UI CÓ HÀNH VI

◈ UI tĩnh vs UI tương tác

- UI tĩnh: chỉ hiển thị
- UI tương tác: **phản hồi hành động**

◈ Các widget tương tác cơ bản

- `ElevatedButton` → hành động chính
- `IconButton` → hành động phụ
- `GestureDetector` → bắt tap mọi widget

◈ Callback là gì?

- Function được gọi khi có sự kiện
- Ví dụ:
  - `onTap`
  - `onPressed`

👉 UI **phát sinh sự kiện**, không xử lý logic phức tạp.

VII. STATE & SETSTATE (MỨC UI CƠ BẢN)

◈ Khi nào cần StatefulWidget?

- Khi UI:
  - thay đổi khi click
  - đổi trạng thái hiển thị

◈ setState làm gì?

- Báo Flutter:
  "UI này cần vẽ lại"

⚠ Sai lầm phổ biến

- Lạm dụng `setState`
- Xử lý logic dài trong UI

🧠 **Nguyên tắc:**
Slot 6 → `setState` **chỉ để đổi UI**, không phải xử lý nghiệp vụ.

VIII. TÁCH WIDGET & CLEAN UI CODE

◈ Khi nào nên tách widget?

- Item list
- Button tái sử dụng
- UI lặp lại

◈ Lợi ích

- Dễ đọc
- Dễ sửa
- Dễ test

🗀 Gợi ý cấu trúc

```
screens/
  home_screen.dart
widgets/
```


PhucPT10@fpt.edu.vn

`product_item.dart`

IX. HOT RELOAD & DEBUG UI

◈ Hot Reload vs Hot Restart

- Reload: giữ state
- Restart: reset app

◈ Khi UI không đổi dù đã sửa code?

- Stateful giữ state
- Widget không rebuild

◈ Debug UI

- Layout lỗi → kiểm tra widget cha
- Click không chạy → kiểm tra callback

X. LỖI UI PHỔ BIẾN SLOT 6

◈ RenderFlex overflow

- Nguyên nhân: Column + content dài
- Sửa: ListView / Expanded

◈ ListView trong Column

- Nguyên nhân: không giới hạn height
- Sửa: Expanded / Flexible

◈ Button không nhận tap

- Nguyên nhân: callback null
- Sửa: truyền function

XI. TỔNG KẾT & ĐỊNH HƯỚNG

- UI Flutter:
  - Screen rõ ràng
  - List đúng công cụ
  - Interaction đơn giản – dễ hiểu

👉 Chuẩn bị cho:

- Navigation
- State nâng cao
- Reactive UI

B PHẦN B. DART PHỤC VỤ UI (GIỚI HẠN PHẠM VI)

I. LIST & DATA FOR UI

- `List` → nguồn dữ liệu UI
- `map()` → chuyển data → widget

II. FUNCTION & CALLBACK

- Function truyền vào widget
- UI gọi callback khi có sự kiện

III. ENUM TRONG UI

- Dùng cho trạng thái:
  - loading
  - done
  - error
- Tránh dùng string rời rạc

PhucPT10@fpt.edu.vn

KẾT LUẬN CHO SINH VIÊN

"UI Flutter không phải là vẽ cho chạy
mà là **thiết kế giao diện có cấu trúc – có hành vi – có thể mở rộng**"

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. Kỹ thuật: Scaffold & Screen UI

Kinh nghiệm thực tế khi dùng trong app Flutter

- Xuất hiện ở:
  - Home Screen
  - List Screen
  - Detail Screen
- Trong app thật:
  - **Mỗi màn hình là 1 Scaffold riêng**
  - Không có chuyện "1 Scaffold cho cả app"

Bối cảnh thực tế

App sinh viên có:

- Màn hình danh sách môn học
- Màn hình chi tiết môn học

Vấn đề UI gặp phải

- Sinh viên dùng **1 Scaffold cho tất cả**
- Thay đổi body liên tục → code rối, khó debug

Vì sao Scaffold là lựa chọn đúng

- Scaffold giúp:
  - cô lập UI từng screen
  - dễ gắn navigation sau này

Lỗi thường gặp

Lỗi 1: Không dùng Scaffold cho screen

- **Biểu hiện**:
  - AppBar hiển thị sai
  - Button không có hiệu ứng Material
- **Nguyên nhân gốc rễ**:
  - Nghĩ Scaffold "không bắt buộc"
- **Khắc phục**:
  - Mỗi screen chính → luôn có Scaffold
- **Phòng tránh**:
  - Tự hỏi: *"Đây có phải 1 màn hình không?"*

Lỗi 2: Lồng nhiều Scaffold

- **Biểu hiện**:
  - UI lạ, AppBar chồng AppBar
- **Nguyên nhân**:
  - Không phân biệt screen vs widget con
- **Khắc phục**:
  - Widget con **không có Scaffold**
- **Phòng tránh**:
  - Scaffold chỉ ở cấp screen

Tips & Tricks

- Đặt tên file:
  - `home_screen.dart`
  - `product_list_screen.dart`
- Nhìn file là biết **1 screen**

Best Practices (Slot 6)

- ✅ 1 screen = 1 Scaffold

PhucPT10@fpt.edu.vn

- ❌ Không xử lý logic trong Scaffold
- ✅ Scaffold chỉ lo layout

🎓 Bài học rút ra

Nếu chỉ nhớ 1 điều:

Scaffold không phải widget trang trí – nó là ranh giới của 1 màn hình

⚠️ **Cảnh báo tư duy sai**

"Không có Scaffold vẫn chạy" → **SAI trong app thật**

🔷 2. Kỹ thuật: ListView / ListView.builder

☑️ Kinh nghiệm thực tế

- Xuất hiện ở:
  - danh sách sản phẩm
  - danh sách sinh viên
  - danh sách công việc
- Trong app thật:
  - **Gần như luôn dùng ListView.builder**

📍 **Bối cảnh**

App todo có 30–50 task

❗ **Vấn đề**

- Dùng `Column` → overflow, lag

👉 **Vì sao ListView.builder đúng**

- Render theo nhu cầu
- Scroll mượt
- Dữ liệu lớn không vỡ UI

⚠️ Lỗi thường gặp

❌ *Lỗi 1: Dùng Column cho danh sách*

- **Biểu hiện**:
  - RenderFlex overflow
- **Nguyên nhân**:
  - Tư duy web / HTML
- **Ví dụ sai**:

```
Column(
  children: tasks.map((t) => Text(t)).toList(),
)
```

- **Khắc phục**:
  - Đổi sang ListView.builder
- **Phòng tránh**:
  - Hễ là danh sách → nghĩ ListView

❌ *Lỗi 2: ListView trong Column không bọc Expanded*

- **Biểu hiện**:
  - App crash / layout lỗi
- **Nguyên nhân**:
  - ListView không có height
- **Khắc phục**:

```
Expanded(
  child: ListView.builder(...),
)
```


PhucPT10@fpt.edu.vn

Tips & Tricks

- `itemBuilder` **chỉ render UI**
- Đừng xử lý logic ở đây

Best Practices

- ✅ Dữ liệu động → ListView.builder
- ❌ Không dùng Column cho list dài
- ✅ Tách item thành widget riêng

Bài học rút ra

Danh sách trong app thật **luôn gắn với dữ liệu**, không phải UI cứng.

⚠️ **Cảnh báo tư duy sai**

"Danh sách ít thì dùng Column" → **dễ thành thói quen xấu**

3. Kỹ thuật: Card & ListTile

✅ Kinh nghiệm thực tế

- Xuất hiện ở:
  - list sản phẩm
  - list người dùng
- Trong app thật:
  - Card + ListTile = combo chuẩn

📍 **Bối cảnh**

Danh sách sinh viên

❗ **Vấn đề**

- UI tự ghép Row → rối, khó đọc

👉 **Vì sao ListTile đúng**

- Layout chuẩn
- Nhất quán
- Ít lỗi spacing

⚠️ Lỗi thường gặp

❌ *Lỗi: Tự layout bằng Row + Padding*

- **Biểu hiện**:
  - UI mỗi item mỗi kiểu
- **Nguyên nhân**:
  - Không biết ListTile
- **Khắc phục**:
  - Ưu tiên ListTile

Tips

- Nếu item có:
  - icon + text + action → nghĩ ListTile

Best Practices

- ✅ ListTile cho danh sách chuẩn
- ✅ Card để phân tách item
- ❌ Không "vẽ tay" layout khi chưa cần

Bài học rút ra

UI đẹp trong app thật = **nhất quán**, không phải phức tạp.

4. Kỹ thuật: Interactive Widgets (Button, GestureDetector)

Kinh nghiệm thực tế

- UI app thật:
  - item bấm được
  - nút phản hồi rõ ràng

Bối cảnh

Click item để xem chi tiết

Vấn đề

- Click không phản hồi
- Không biết item nào được click

Vai trò kỹ thuật

- `onTap`, `onPressed` là **cầu nối UI ↔ hành vi**

Lỗi thường gặp

Lỗi 1: Quên callback

- **Biểu hiện**:
  - Nút bấm không chạy
- **Nguyên nhân**:
  - `onPressed: null`
- **Khắc phục**:
  - Luôn truyền function

Lỗi 2: Viết logic dài trong onTap

- **Biểu hiện**:
  - Code UI rối
- **Nguyên nhân**:
  - Chưa biết tách function
- **Khắc phục**:

```
onTap: () => onItemSelected(item);
```

Tips

- UI chỉ **gọi hàm**
- Logic để riêng

Best Practices

- ✅ GestureDetector cho item
- ✅ Button cho hành động chính
- ❌ Không xử lý nghiệp vụ trong UI

Bài học rút ra

UI tốt là UI **phản hồi rõ ràng**, không phải UI có nhiều nút.

5. Kỹ thuật: StatefulWidget & setState (UI cơ bản)

Kinh nghiệm thực tế

- Dùng khi:
  - đổi icon
  - highlight item
  - toggle trạng thái

Bối cảnh

Click item → đổi màu

PhucPT10@fpt.edu.vn

⚠ Lỗi thường gặp

✕ *Lỗi: Lạm dụng setState*

- **Biểu hiện**:
  - UI rebuild toàn màn hình
- **Nguyên nhân**:
  - Chưa biết quản lý state
- **Khắc phục**:
  - setState **chỉ cho UI nhỏ**

🛡 Best Practices

- ✅ setState = đổi hiển thị
- ❌ Không xử lý logic phức tạp

🎓 Bài học rút ra

setState không xấu, **lạm dụng mới xấu**

◈ 6. Hot Reload & Debug UI

⚠ Lỗi sinh viên hay gặp

- Code sửa nhưng UI không đổi

🔍 Nguyên nhân:

- StatefulWidget giữ state
- Cần Hot Restart

🛠 Mẹo debug:

- In `print()` trong build
- Kiểm tra callback có chạy không

🎯 TỔNG KẾT CHO SINH VIÊN

Sau Slot 6, sinh viên phải **nhận ra**:

- ❌ UI demo chạy được ≠ UI app thật
- ✅ App thật cần:
  - screen rõ ràng
  - list đúng công cụ
  - interaction đơn giản, dễ hiểu

Nếu chỉ nhớ 3 điều:

1. 1 screen = 1 Scaffold
2. Danh sách = ListView.builder
3. UI chỉ hiển thị & phát sinh sự kiện

👉 Đây là **nền móng bắt buộc** trước khi học:

- Navigation
- State nâng cao
- App nhiều màn hình

PhucPT10@fpt.edu.vn

EXERCISE

📝 BÀI TẬP TỔNG HỢP 1

STUDENT LIST INTERACTION APP

1 TỔNG QUAN DỰ ÁN

◈ Tên dự án

Student List Interaction App

◈ Mô tả ngắn gọn

Ứng dụng Flutter hiển thị **danh sách sinh viên** bằng `ListView.builder`,
mỗi sinh viên được hiển thị bằng **Card + ListTile**,
cho phép người dùng **tap chọn / đánh dấu trạng thái** trực tiếp trên UI.

👉 Đây là dạng UI **xuất hiện trong hầu hết app thật**:

- danh sách sản phẩm
- danh sách người dùng
- danh sách công việc

◈ Mục tiêu học tập

Sau dự án này, sinh viên sẽ:

- Biết xây dựng **ListView đúng chuẩn**
- Biết **tách Card/ListTile thành widget tái sử dụng**
- Hiểu rõ **UI hiển thị ≠ UI tương tác**
- Biết dùng `setState` **đúng mức độ UI local**
- Tránh các lỗi UI rất phổ biến của người mới

◈ Đối tượng phù hợp

- Sinh viên vừa hoàn thành **Slot 6**
- Nền tảng Dart & Flutter cơ bản
- Cần **làm được app UI thật**, không chỉ chạy ví dụ

2 CHỨC NĂNG CHÍNH CỦA ỨNG DỤNG

Ứng dụng gồm **1 màn hình chính (Home Screen)**:

- Hiển thị danh sách sinh viên bằng `ListView.builder`
- Mỗi item:
  - Hiển thị tên + mã sinh viên
  - Được bọc trong **Card**
  - Sử dụng **ListTile**
- Có tương tác:
  - Tap vào item → đổi trạng thái (đã chọn / chưa chọn)
  - Icon thay đổi theo trạng thái

👉 Mỗi chức năng **gắn trực tiếp** với kiến thức Slot 6

👉 Không có navigation phức tạp, không kiến trúc nâng cao

3 CÔNG NGHỆ & CÔNG CỤ SỬ DỤNG

- Flutter SDK
- Dart
- Android Studio hoặc VS Code
- Android Emulator
- Material Widgets (mặc định Flutter)

4 CẤU TRÚC THƯ MỤC DỰ ÁN (CHUẨN UI LIST)

```
lib/
├── main.dart
```


PhucPT10@fpt.edu.vn

```
├── models/
│   └── student.dart
├── screens/
│   └── home_screen.dart
├── widgets/
│   └── student_card.dart
```

📌 Giải thích sư phạm

- `screens/`
  👉 **Chỉ lo tổ chức màn hình (Scaffold, ListView)**
- `widgets/`
  👉 **Chỉ hiển thị 1 item (Card / ListTile)**
- ❌ Không viết Card trực tiếp trong `ListView.builder`

5 DANH SÁCH CÁC BƯỚC THỰC HIỆN (TÓM TẮT)

1. Tạo project Flutter
2. Tạo model dữ liệu `Student`
3. Chuẩn bị danh sách sinh viên mẫu
4. Tạo Home Screen với Scaffold
5. Xây dựng ListView.builder
6. Tạo widget StudentCard (Card + ListTile)
7. Truyền dữ liệu vào StudentCard
8. Thêm tương tác tap
9. Xử lý `setState` cho UI
10. Kiểm tra & hoàn thiện UI

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (QUAN TRỌNG NHẤT)

◈ BƯỚC 1: TẠO PROJECT FLUTTER

📌 *Mục tiêu*

Khởi tạo đúng môi trường Flutter để tránh lỗi vặt UI.

🛠 *Thao tác*

- Mở Android Studio / VS Code
- Tạo project mới:
- `flutter create student_list_app`

✅ *Kiểm tra*

- Chạy `flutter run`
- Thấy app Flutter mặc định chạy OK

◈ BƯỚC 2: TẠO MODEL DỮ LIỆU

📌 *Mục tiêu*

Hiểu: **ListView luôn render từ dữ liệu → không hardcode UI**

🛠 *Thao tác*

- Tạo file: `lib/models/student.dart`

```
class Student {
  final String id;
  final String name;
  bool isSelected;

  Student({
    required this.id,
    required this.name,
```


PhucPT10@fpt.edu.vn

```
    this.isSelected = false,
  });
}
```

Giải thích

- `isSelected` dùng để **đổi trạng thái UI**
- Đây là **state đơn giản phục vụ UI**, chưa phải logic phức tạp

BƯỚC 3: CHUẨN BỊ DỮ LIỆU MẪU

Mục tiêu

Kết nối **Dart List → UI ListView**

Thao tác

- Mở `home_screen.dart`
- Tạo danh sách mẫu:

```
final List<Student> students = [
  Student(id: 'SV01', name: 'Nguyễn Văn A'),
  Student(id: 'SV02', name: 'Trần Thị B'),
  Student(id: 'SV03', name: 'Lê Văn C'),
];
```

Lỗi thường gặp

- Dùng `List<dynamic>` → khó kiểm soát UI

BƯỚC 4: TẠO HOME SCREEN VỚI SCAFFOLD

Mục tiêu

Củng cố tư duy: **1 Screen = 1 Scaffold**

Thao tác

```
class HomeScreen extends StatefulWidget {
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Student List'),
      ),
      body: Container(),
    );
  }
}
```

Kiểm tra

- App có AppBar hiển thị đúng

BƯỚC 5: XÂY DỰNG LISTVIEW.BUILDER

Mục tiêu

Hiểu vì sao **Column không dùng cho danh sách dài**

Thao tác

```
body: ListView.builder(
  itemCount: students.length,
  itemBuilder: (context, index) {
    return Text(students[index].name);
```


PhucPT10@fpt.edu.vn

```
  },
),
```

⚠ *Lỗi thường gặp*

- Quên `itemCount`
- Dùng `Column` gây overflow

◈ BƯỚC 6: TẠO STUDENT CARD WIDGET RIÊNG

📌 *Mục tiêu*

Rèn tư duy **tách widget – code dễ đọc**

🛠 *Thao tác*

- Tạo file: `lib/widgets/student_card.dart`

```
class StudentCard extends StatelessWidget {
  final Student student;
  final VoidCallback onTap;

  const StudentCard({
    required this.student,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        title: Text(student.name),
        subtitle: Text(student.id),
        trailing: Icon(
          student.isSelected ? Icons.check_circle :
Icons.circle_outlined,
        ),
        onTap: onTap,
      ),
    );
  }
}
```

◈ BƯỚC 7: KẾT NỐI STUDENTCARD VÀO LISTVIEW

```
return StudentCard(
  student: students[index],
  onTap: () {
    setState(() {
      students[index].isSelected = !students[index].isSelected;
    });
  },
);
```

📌 *Giải thích*

- `setState` **chỉ đổi UI**
- Không xử lý logic phức tạp

PhucPT10@fpt.edu.vn

CODE HOÀN CHỈNH

lib/models/student.dart

```
class Student {
  final String id;
  final String name;
  bool isSelected;

  Student({
    required this.id,
    required this.name,
    this.isSelected = false,
  });
}
```

lib/widgets/student_card.dart

```
import 'package:flutter/material.dart';
import '../models/student.dart';

class StudentCard extends StatelessWidget {
  final Student student;
  final VoidCallback onTap;

  const StudentCard({
    Key? key,
    required this.student,
    required this.onTap,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: ListTile(
        leading: CircleAvatar(
          child: Text(
            student.name.isNotEmpty ? student.name[0] : '?',
          ),
        ),
        title: Text(
          student.name,
          style: const TextStyle(fontSize: 16, fontWeight: FontWeight.w500),
        ),
        subtitle: Text(
          student.id,
          style: const TextStyle(color: Colors.grey),
        ),
        trailing: Icon(
          student.isSelected ? Icons.check_circle : Icons.circle_outlined,
          color: student.isSelected ? Colors.green : Colors.grey,
        ),
        onTap: onTap,
      ),
    );
  }
}
```

lib/screens/home_screen.dart

```
import 'package:flutter/material.dart';
import '../models/student.dart';
import '../widgets/student_card.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({Key? key}) : super(key: key);

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  // BƯỚC 3: Chuẩn bị danh sách sinh viên mẫu
  final List<Student> students = [
    Student(id: 'SV01', name: 'Nguyễn Văn A'),
    Student(id: 'SV02', name: 'Trần Thị B'),
    Student(id: 'SV03', name: 'Lê Văn C'),
    Student(id: 'SV04', name: 'Phạm Văn D'),
    Student(id: 'SV05', name: 'Hoàng Thị E'),
  ];

  @override
  Widget build(BuildContext context) {
    // BƯỚC 4: Tạo Home Screen với Scaffold
    return Scaffold(
      appBar: AppBar(
        title: const Text('Student List'),
        centerTitle: true,
      ),
      // BƯỚC 5: Xây dựng ListView.builder
      body: ListView.builder(
        itemCount: students.length,
        itemBuilder: (context, index) {
          final student = students[index];

          // BƯỚC 7: Kết nối StudentCard vào ListView
          return StudentCard(
            student: student,
            onTap: () {
              // BƯỚC 9: Xử lý setState cho UI
              setState(() {
                student.isSelected = !student.isSelected;
              });
            },
          );
        },
      ),
    );
  }
}
```


PhucPT10@fpt.edu.vn

lib/main.dart

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const StudentListApp());
}

class StudentListApp extends StatelessWidget {
  const StudentListApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Student List Interaction App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: false,
      ),
      home: const HomeScreen(),
    );
  }
}
```

◈ BƯỚC 8: KIỂM TRA TƯƠNG TÁC

✓ Quan sát

- Tap vào item
- Icon thay đổi ngay
- UI phản hồi rõ ràng

7 KIỂM TRA & CHẠY ỨNG DỤNG

- Bật emulator
- `flutter run`
- Kiểm tra:
  - List scroll được
  - Card hiển thị đều
  - Tap hoạt động đúng

8 CÁC LỖI PHỔ BIẾN & CÁCH XỬ LÝ

Lỗi	Nguyên nhân	Cách sửa
List không scroll	Dùng Column	Dùng ListView
Tap không chạy	Quên callback	Truyền onTap
UI không đổi	Quên setState	Bọc logic trong setState

9 KẾT QUẢ ĐẠT ĐƯỢC SAU DỰ ÁN

Sinh viên có thể:

- Tự xây **1 screen danh sách hoàn chỉnh**
- Hiểu rõ:
  - Vì sao phải dùng ListView.builder
  - Vì sao Card/ListTile nên tách riêng
- Sẵn sàng học:
  - Navigation
  - State nâng cao

PhucPT10@fpt.edu.vn

10 BÀI TẬP MỞ RỘNG

- Thêm nút xoá item
- Highlight item được chọn
- So sánh ListView vs Column + map

PhucPT10@fpt.edu.vn

MỤC TIÊU NÂNG CẤP (RẤT QUAN TRỌNG)

Sau nâng cấp này, sinh viên sẽ **hiểu sâu hơn**:

- Một **item trong List** có thể có:
  - **Trạng thái UI** (selected)
  - **Hành động điều hướng** (xem chi tiết)
- Vì sao:
  - **Card / ListTile KHÔNG nên tự navigation**
  - **Screen mới là nơi quyết định navigation**
- Tư duy UI:
  "Tap không chỉ là đổi màu – tap còn là mở ngữ cảnh mới"

1 THAY ĐỔI TỔNG QUAN CHỨC NĂNG

Trước

- Tap item → đổi trạng thái (icon)

Sau (Nâng cấp)

Mỗi item sinh viên có **2 dạng tương tác rõ ràng**:

Vị trí	Hành vi
Tap **toàn Card**	Chuyển sang **Student Detail Screen**
Tap **icon trailing**	Toggle trạng thái `selected`

👉 Đây là pattern UI **rất phổ biến trong app thật**

2 CẬP NHẬT MODEL STUDENT (ĐẦY ĐỦ THÔNG TIN)

Mục tiêu sư phạm

- Hiểu: **Detail Screen luôn cần dữ liệu đầy đủ hơn List**

```
// lib/models/student.dart
class Student {
  final String id;
  final String name;
  final String email;
  final String major;
  bool isSelected;

  Student({
    required this.id,
    required this.name,
    required this.email,
    required this.major,
    this.isSelected = false,
  });
}
```

💡 Giải thích cho sinh viên:

- List chỉ hiển thị **một phần**
- Detail Screen hiển thị **toàn bộ thông tin**

3 THÊM DETAIL SCREEN (SCREEN MỚI)

🗂 Cấu trúc thư mục (NÂNG CẤP)

```
lib/
├── screens/
│   ├── home_screen.dart
│   └── student_detail_screen.dart
```

🔷 StudentDetailScreen

```
// lib/screens/student_detail_screen.dart
import 'package:flutter/material.dart';
import '../models/student.dart';

class StudentDetailScreen extends StatelessWidget {
  final Student student;

  const StudentDetailScreen({
    Key? key,
    required this.student,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Student Detail'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              student.name,
              style: const TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 16),
            Text('Student ID: ${student.id}'),
            const SizedBox(height: 8),
            Text('Email: ${student.email}'),
            const SizedBox(height: 8),
            Text('Major: ${student.major}'),
          ],
        ),
      ),
    );
  }
}
```

📌 Giải thích sư phạm:

- Detail Screen:
  - ❌ Không xử lý list

PhucPT10@fpt.edu.vn

- ❌ Không setState
- ✅ Chỉ hiển thị dữ liệu

4 NÂNG CẤP STUDENT CARD – TÁCH RÕ 2 TƯƠNG TÁC

◈ StudentCard (CẬP NHẬT)

```
// lib/widgets/student_card.dart
class StudentCard extends StatelessWidget {
  final Student student;
  final VoidCallback onSelect;
  final VoidCallback onOpenDetail;

  const StudentCard({
    Key? key,
    required this.student,
    required this.onSelect,
    required this.onOpenDetail,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: ListTile(
        onTap: onOpenDetail, // 👉 MỞ CHI TIẾT
        leading: CircleAvatar(
          child: Text(student.name[0]),
        ),
        title: Text(student.name),
        subtitle: Text(student.id),
        trailing: IconButton(
          icon: Icon(
            student.isSelected
                ? Icons.check_circle
                : Icons.circle_outlined,
            color: student.isSelected ? Colors.green : Colors.grey,
          ),
          onPressed: onSelect, // 👉 CHỈ ĐỔI TRẠNG THÁI
        ),
      ),
    );
  }
}
```

⚠️ Điểm dạy rất quan trọng:

- **Không dùng onTap cho cả icon**
- **Icon có hành vi riêng → IconButton**

5 HOME SCREEN – KẾT NỐI NAVIGATION

```
// lib/screens/home_screen.dart
import 'package:flutter/material.dart';
import '../models/student.dart';
import '../widgets/student_card.dart';
import 'student_detail_screen.dart';
```


PhucPT10@fpt.edu.vn

```
class HomeScreen extends StatefulWidget {
  const HomeScreen({Key? key}) : super(key: key);

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final List<Student> students = [
    Student(
      id: 'SV01',
      name: 'Nguyễn Văn A',
      email: 'a@student.edu',
      major: 'Computer Science',
    ),
    Student(
      id: 'SV02',
      name: 'Trần Thị B',
      email: 'b@student.edu',
      major: 'Information Systems',
    ),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Student List'),
        centerTitle: true,
      ),
      body: ListView.builder(
        itemCount: students.length,
        itemBuilder: (context, index) {
          final student = students[index];

          return StudentCard(
            student: student,
            onSelect: () {
              setState(() {
                student.isSelected = !student.isSelected;
              });
            },
            onOpenDetail: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (_) =>
                      StudentDetailScreen(student: student),
                ),
              );
            },
          );
        },
      ),
    );
```


PhucPT10@fpt.edu.vn

```
  }
}
```

6 CHECKPOINT SƯ PHẠM (BẮT BUỘC GIẢI THÍCH CHO SINH VIÊN)

Sinh viên **phải trả lời được**:

1. ❓ Vì sao **navigation không viết trong StudentCard**
2. ❓ Vì sao **icon có onPressed riêng**
3. ❓ Vì sao DetailScreen không cần StatefulWidget
4. ❓ Vì sao List chỉ hiển thị ít thông tin

CODE HOÀN CHỈNH:

lib/models/student.dartv2

```
class Student {
  class Student {
  final String id;
  final String name;
  final String email;
  final String major;
  bool isSelected;

  Student({
    required this.id,
    required this.name,
    required this.email,
    required this.major,
    this.isSelected = false,
  });
}
```

lib/screens/student_detail_screen.dart

```
import 'package:flutter/material.dart';
import '../models/student.dart';

class StudentDetailScreen extends StatelessWidget {
  final Student student;

  const StudentDetailScreen({
    Key? key,
    required this.student,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Student Detail'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              student.name,
              style: const TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
```


PhucPT10@fpt.edu.vn

```
            ),
            const SizedBox(height: 16),
            Text('Student ID: ${student.id}'),
            const SizedBox(height: 8),
            Text('Email: ${student.email}'),
            const SizedBox(height: 8),
            Text('Major: ${student.major}'),
            const SizedBox(height: 16),
            Row(
              children: [
                const Text(
                  'Selected status: ',
                  style: TextStyle(fontWeight: FontWeight.w500),
                ),
                Icon(
                  student.isSelected
                      ? Icons.check_circle
                      : Icons.circle_outlined,
                  color: student.isSelected ? Colors.green : Colors.grey,
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/widgets/student_card.dartv2

```
import 'package:flutter/material.dart';
import '../models/student.dart';

class StudentCard extends StatelessWidget {
  final Student student;
  final VoidCallback onSelect;
  final VoidCallback onOpenDetail;

  const StudentCard({
    Key? key,
    required this.student,
    required this.onSelect,
    required this.onOpenDetail,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: ListTile(
        onTap: onOpenDetail, // 👉 MỞ CHI TIẾT
        leading: CircleAvatar(
          child: Text(
            student.name.isNotEmpty ? student.name[0] : '?',
          ),
        ),
        title: Text(student.name),
        subtitle: Text(student.id),
        trailing: IconButton(
          icon: Icon(
```


PhucPT10@fpt.edu.vn

```
            student.isSelected ? Icons.check_circle : Icons.circle_outlined,
            color: student.isSelected ? Colors.green : Colors.grey,
          ),
          onPressed: onSelect, // 👉 CHỈ ĐỔI TRẠNG THÁI
        ),
      ),
    );
  }
}
```

lib/screens/home_screen.dartv2

```
import 'package:flutter/material.dart';
import '../models/student.dart';
import '../widgets/student_card.dart';
import 'student_detail_screen.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({Key? key}) : super(key: key);

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  // Lấy dữ liệu mẫu từ Student model (đầy đủ field)
  final List<Student> students = [
    Student(
      id: 'SV01',
      name: 'Nguyễn Văn A',
      email: 'a@student.edu',
      major: 'Computer Science',
    ),
    Student(
      id: 'SV02',
      name: 'Trần Thị B',
      email: 'b@student.edu',
      major: 'Information Systems',
    ),
    Student(
      id: 'SV03',
      name: 'Lê Văn C',
      email: 'c@student.edu',
      major: 'Software Engineering',
    ),
    Student(
      id: 'SV04',
      name: 'Phạm Văn D',
      email: 'd@student.edu',
      major: 'Data Science',
    ),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Student List'),
        centerTitle: true,
      ),
      body: ListView.builder(
```

```
        itemCount: students.length,
        itemBuilder: (context, index) {
          final student = students[index];

          return StudentCard(
            student: student,
            onSelect: () {
              setState(() {
                student.isSelected = !student.isSelected;
              });
            },
            onOpenDetail: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (_) => StudentDetailScreen(student: student),
                ),
              );
            },
          );
        },
      ),
    );
  }
}
```

lib/main.dartv2

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const StudentListApp());
}

class StudentListApp extends StatelessWidget {
  const StudentListApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Student List Interaction App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: false,
      ),
      home: const HomeScreen(),
    );
  }
}
```

Cách tích hợp vào project

1. Giữ nguyên cấu trúc thư mục:

```Text
lib/
├── main.dart
├── models/
```


PhucPT10@fpt.edu.vn

```
│   └── student.dart
├── screens/
│   ├── home_screen.dart
│   └── student_detail_screen.dart
└── widgets/
    └── student_card.dart
```

2. Đảm bảo:

    - `student.dart` đã có đủ các field: `id`, `name`, `email`, `major`, `isSelected`.
    - `HomeScreen` import đúng `student_detail_screen.dart` và `student_card.dart`.
    - Navigation chỉ viết trong `HomeScreen`, **không** viết trong `StudentCard`.

PhucPT10@fpt.edu.vn

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

Thuật ngữ	Định nghĩa dễ hiểu	Dùng trong Slot 6	Hiểu sai thường gặp (Cảnh báo)
Scaffold	Khung bố cục cơ bản của 1 màn hình Flutter	Tạo AppBar, body chứa ListView	Dùng Scaffold cho mọi widget nhỏ → UI rối
Screen	Một màn hình hoàn chỉnh (thường là 1 page)	HomeScreen quản lý layout tổng	Nhồi hết logic + UI vào 1 screen
Widget Tree	Cấu trúc widget lồng nhau tạo UI	ListView → Card → ListTile → Text	Lồng quá sâu làm UI khó đọc
StatelessWidget	Widget không tự thay đổi trạng thái	Card, ItemTile hiển thị dữ liệu	Nghĩ rằng "stateless = không tương tác"
StatefulWidget	Widget có trạng thái UI thay đổi được	Screen quản lý selected item	Dùng StatefulWidget cho mọi thứ
setState()	Báo Flutter rebuild UI khi state đổi	Cập nhật selected / liked	Gọi setState quá nhiều gây khó debug
ListView	Widget hiển thị danh sách cuộn	Hiển thị danh sách item	Dùng ListView cho danh sách nhỏ
ListView.builder	ListView sinh item động theo dữ liệu	Danh sách lớn / dữ liệu động	Dùng builder nhưng viết logic phức tạp
Item	Một phần tử trong danh sách	1 sản phẩm / task / sinh viên	Viết UI item trực tiếp trong ListView
Card	Widget hiển thị khối thông tin có viền	Bao bọc item trong list	Copy–paste Card ở nhiều nơi
ListTile	Widget layout sẵn cho item list	title, subtitle, leading	Custom quá nhiều làm mất lợi thế
Reusable Widget	Widget viết 1 lần dùng nhiều nơi	ItemCard, ItemTile	Viết nhưng không tái sử dụng
Callback	Hàm được truyền vào để xử lý sự kiện	onTap, onPressed	Viết logic lớn trong callback
onTap / onPressed	Sự kiện khi người dùng tương tác	Tap item, bấm nút	Xử lý quá nhiều việc trong UI
Interactive Widget	Widget cho phép người dùng tương tác	InkWell, IconButton	Nhầm Interactive = Animation
InkWell	Widget bắt tap có hiệu ứng ripple	Tap toàn bộ Card	Dùng sai vị trí → không bắt tap
GestureDetector	Bắt các gesture (tap, long press…)	Custom tương tác	Dùng cho mọi tap đơn giản
UI State (Local)	Trạng thái chỉ ảnh hưởng 1 screen	selected, expanded	Lưu state rải rác khó kiểm soát
Data Model	Class mô tả dữ liệu hiển thị	ItemModel	Trộn model với UI
List (Dart)	Danh sách dữ liệu	List<ItemModel>	Thao tác trực tiếp làm UI lỗi
Control Flow	if / for điều khiển hiển thị UI	Hiện icon, trạng thái	Viết logic rối trong build()
UI Consistency	UI đồng bộ toàn app	Card giống nhau	Mỗi screen 1 kiểu
Maintainability	Dễ sửa, dễ bảo trì	Tách widget	Code chạy được là đủ
Scalability (UI)	UI chịu được mở rộng	Thêm action, trạng thái	UI chỉ đúng cho hiện tại
Rebuild	Flutter vẽ lại UI	Sau setState	Nghĩ rebuild = lỗi
Overflow	UI tràn màn hình	Text dài trong Card	Fix bằng SizedBox cứng
Layout Issue	Lỗi bố cục UI	Sai padding, alignment	Debug bằng thử–sai

PhucPT10@fpt.edu.vn

NHÓM 1. SCREEN & UI STRUCTURE

Thuật ngữ	Giải thích dễ hiểu	Dùng trong app thật ở đâu?	Hiểu sai thường gặp
Screen	Một màn hình hoàn chỉnh trong app	Home, List, Detail screen	Nghĩ screen = file
Scaffold	Khung tiêu chuẩn của 1 screen Flutter	Mỗi màn hình chính	Dùng nhiều Scaffold lồng nhau
AppBar	Thanh tiêu đề trên cùng của screen	Tên màn hình, action	Tạo AppBar ngoài Scaffold
Body	Khu vực hiển thị nội dung chính	List, form, content	Nhét quá nhiều widget trực tiếp
MaterialApp	Gốc của toàn bộ app Material	App root	Nhầm vai trò với Scaffold

NHÓM 2. WIDGET TREE & COMPOSITION

Thuật ngữ	Giải thích dễ hiểu	Ý nghĩa thực tế	Hiểu sai thường gặp
Widget Tree	Cấu trúc cây các widget lồng nhau	Hiểu layout & debug UI	Không hình dung được tree
Root Widget	Widget gốc của một khu vực UI	Body chỉ có 1 root	Nghĩ body nhận nhiều widget
Composition	Ghép UI bằng widget con	Xây UI linh hoạt	Ghép Row/Column rối
Nested Widget	Widget lồng nhiều cấp	UI phức tạp	Tree quá sâu, khó đọc

NHÓM 3. LISTVIEW & DANH SÁCH

Thuật ngữ	Giải thích dễ hiểu	Dùng khi nào	Hiểu sai thường gặp
ListView	Widget hiển thị danh sách scroll	Danh sách ngắn	Dùng cho dữ liệu lớn
ListView.builder	Tạo item khi cần	Danh sách động, dài	Không hiểu itemBuilder
itemBuilder	Hàm tạo UI cho từng item	Render từng dòng	Viết logic phức tạp trong đây
itemCount	Số lượng item	Giới hạn list	Quên itemCount gây lỗi
Scroll	Cuộn danh sách	UX cơ bản	Dùng Column gây overflow

NHÓM 4. CARD & LISTTILE

Thuật ngữ	Giải thích dễ hiểu	Vai trò UX	Hiểu sai thường gặp
Card	Khung hiển thị nội dung	Phân tách item	Lạm dụng Container
ListTile	Layout chuẩn cho item list	UI nhanh, đồng nhất	Tự ghép Row phức tạp
leading	Nội dung bên trái	Icon, avatar	Đặt widget quá lớn
title	Nội dung chính	Tên item	Dùng sai cấp text
subtitle	Nội dung phụ	Mô tả ngắn	Nhét text dài
trailing	Nội dung bên phải	Icon, action	Nhét logic vào đây

PhucPT10@fpt.edu.vn

NHÓM 5. INTERACTIVE WIDGETS & CALLBACK

Thuật ngữ	Giải thích dễ hiểu	Dùng trong app thật	Hiểu sai thường gặp
Interactive Widget	Widget có phản hồi người dùng	Button, item click	Nghĩ UI chỉ để hiển thị
ElevatedButton	Nút bấm chuẩn Material	Action chính	Quên onPressed
IconButton	Nút chỉ có icon	Toolbar, list item	Viết logic quá nhiều
GestureDetector	Bắt sự kiện chạm	Custom interaction	Dùng thay Button không cần thiết
onTap / onPressed	Callback khi người dùng thao tác	Xử lý click	Viết toàn bộ logic tại đây
Callback	Hàm được gọi khi có sự kiện	Kết nối UI ↔ hành vi	Không hiểu luồng dữ liệu

NHÓM 6. STATE & UI REBUILD (CƠ BẢN)

Thuật ngữ	Giải thích dễ hiểu	Ý nghĩa thực tế	Hiểu sai thường gặp
StatelessWidget	UI không đổi theo thời gian	Màn hình tĩnh	Dùng cho UI cần tương tác
StatefulWidget	UI có thể thay đổi	Click, chọn item	Dùng cho mọi screen
State	Trạng thái ảnh hưởng UI	Selected, visible	Nhầm với logic nghiệp vụ
setState	Báo Flutter rebuild UI	Đổi hiển thị	Lạm dụng cho logic lớn
Rebuild	UI vẽ lại	Cập nhật giao diện	Nghĩ rebuild = chạy lại app

NHÓM 7. DEBUG & DEVELOPMENT TOOL

Thuật ngữ	Giải thích dễ hiểu	Khi dùng	Hiểu sai thường gặp
Hot Reload	Cập nhật UI nhanh	Chỉnh layout	Không cập nhật state
Hot Restart	Khởi động lại app	Sửa logic	Nhầm với Hot Reload
RenderFlex overflow	Lỗi layout tràn	UI bị cắt	Không hiểu nguyên nhân
Emulator	Thiết bị ảo	Test UI	Nghĩ lỗi do máy

NHÓM 8. DART PHỤC VỤ UI

Thuật ngữ	Giải thích dễ hiểu	Dùng trong UI	Hiểu sai thường gặp
List	Danh sách dữ liệu	Render ListView	Dùng sai kiểu
map()	Chuyển data → UI	Render widget	Nhầm với ListView
Function	Hàm xử lý	Callback UI	Viết logic lớn
Enum	Tập giá trị cố định	Trạng thái UI	Dùng string tùy tiện

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

◈ Prompt 1 – Phân tích vai trò Scaffold trong app thật

Một screen Flutter được xây dựng chỉ bằng `Column`, `Container`, `Text` mà **không sử dụng Scaffold**.
Hãy phân tích:

- Scaffold thực sự đang giải quyết những vấn đề UI nào trong app Flutter?
- UI có thể "chạy được" nhưng **những rủi ro nào sẽ xuất hiện** khi:
  - thêm AppBar
  - thêm SnackBar
  - thêm FloatingActionButton?
- Trong app thật, các lỗi này thường **lộ ra ở giai đoạn nào** (demo, test, production)?

👉 Kết luận:
Scaffold là tiện lợi hay là cấu trúc bắt buộc? Vì sao?

◈ Prompt 2 – Đánh giá Widget Tree & khả năng maintain

Cho một UI có cấu trúc:

- Nhiều `Column` lồng `Row`
- Padding được đặt rải rác
- Một widget dài > 200 dòng

Hãy phân tích:

- Widget Tree hiện tại phản ánh **ý đồ UI rõ ràng hay không**?
- Dấu hiệu nào cho thấy UI này:
  - khó đọc
  - khó sửa
  - dễ vỡ layout khi thêm tính năng?
- Nếu bạn là reviewer, **điểm nào bạn sẽ cảnh báo đầu tiên**?

👉 Không đề xuất kiến trúc mới, chỉ phân tích **vấn đề từ UI hiện tại**.

◈ Prompt 3 – ListView vs Column: Đánh giá quyết định kỹ thuật

Một screen hiển thị danh sách item, ban đầu chỉ có 5–6 phần tử, được xây dựng bằng `Column + children`.
Hãy phân tích:

- Vì sao quyết định này **có thể đúng ở demo** nhưng **sai trong app thật**?
- Những vấn đề nào sẽ xuất hiện khi:
  - số lượng item tăng
  - item có chiều cao khác nhau?
- Dấu hiệu nào cho thấy **đã đến lúc phải đổi sang ListView.builder**?

👉 Trả lời dưới góc nhìn **hiệu năng + maintain UI**.

◈ Prompt 4 – Đánh giá thiết kế Card / ListTile trong ListView

So sánh hai cách tiếp cận:

1. Viết toàn bộ UI Card trực tiếp trong `itemBuilder`
2. Tách Card thành widget riêng (ví dụ: `ProductItem`)

Hãy đánh giá:

- Cách nào **dễ đọc code hơn khi review**?
- Cách nào **ít rủi ro hơn khi thêm interaction** (onTap, selected)?
- Khi app lớn dần, cách nào dễ gây **UI duplication**?

👉 Không hỏi "nên dùng cách nào", hãy **phân tích hệ quả của từng cách**.

PhucPT10@fpt.edu.vn

◈ Prompt 5 – StatelessWidget vs StatefulWidget: Nhận diện lạm dụng state

Một screen được khai báo là `StatefulWidget`, trong đó:

- chỉ thay đổi màu item khi click
- không có dữ liệu phức tạp

Hãy phân tích:

- Những dấu hiệu nào cho thấy `StatefulWidget` đang bị **dùng quá mức**?
- Việc đặt toàn bộ screen trong state sẽ gây khó khăn gì khi:
  - UI dài hơn
  - có nhiều interaction hơn?
- Reviewer nên đặt câu hỏi gì cho người viết code?

◈ Prompt 6 – Đánh giá cách dùng setState ở mức UI

Xét tình huống:

- `setState()` được gọi trong `onPressed`
- Toàn bộ screen bị rebuild
- UI hiện tại còn đơn giản

Hãy phân tích:

- Ở thời điểm hiện tại, cách làm này **có chấp nhận được không**?
- Khi UI phức tạp hơn, rủi ro nào sẽ xuất hiện?
- Dấu hiệu nào cho thấy `setState` đang ảnh hưởng tiêu cực đến UI?

👉 Không đề xuất Provider/Bloc – chỉ đánh giá **chất lượng UI hiện tại**.

◈ Prompt 7 – Interactive Widgets & tư duy callback

Phân tích vai trò của callback (`onTap`, `onPressed`) trong Flutter UI:

- Vì sao callback giúp UI **linh hoạt hơn** so với:
  - hard-code logic trực tiếp trong widget?
- Điều gì xảy ra nếu UI:
  - không truyền callback
  - hoặc xử lý logic dài trong onTap?
- Dấu hiệu nào cho thấy UI đang **trộn lẫn vai trò hiển thị và xử lý**?

◈ Prompt 8 – Đánh giá khả năng mở rộng của UI danh sách

Một màn hình danh sách sản phẩm hiện tại:

- hiển thị Card + ListTile
- click item chỉ in `print()`

Hãy phân tích:

- UI này đã sẵn sàng để thêm:
  - trạng thái selected
  - filter danh sách
  - highlight item hay chưa?
- Những điểm nào trong UI hiện tại sẽ:
  - cản trở mở rộng
  - buộc phải sửa lại nhiều code?

👉 Đánh giá dưới góc nhìn **chuẩn bị cho app thật**.

◈ Prompt 9 – Debug UI từ dấu hiệu bất thường (không báo lỗi)

Xét các hiện tượng:

- Danh sách không scroll
- Item bị cắt nội dung
- Không có error trên console

Hãy phân tích:

- Những nguyên nhân UI **thường gặp nhất** dẫn đến các hiện tượng này

PhucPT10@fpt.edu.vn

- Cách suy luận từng bước để khoanh vùng lỗi
- Vì sao các lỗi này **rất khó với sinh viên mới**?

👉 Không cần code, chỉ cần **lập luận logic debug UI**.

◈ Prompt 10 – Đánh giá mức "sẵn sàng cho dự án thật" của một screen

Hãy đóng vai reviewer và đánh giá một screen Flutter sau Slot 6 theo các tiêu chí:

- Dễ đọc (đọc 5 phút hiểu cấu trúc)
- Dễ sửa (thêm 1 interaction không phá layout)
- Dễ mở rộng (thêm state UI cơ bản)

Hãy trả lời:

- Screen này đã **đủ tiêu chuẩn dùng trong app thật chưa**?
- Nếu chưa, vấn đề lớn nhất nằm ở **tư duy UI hay kỹ thuật UI**?
- Bạn sẽ cảnh báo sinh viên điều gì trước khi cho phép mở rộng tiếp?

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

◈ Prompt 1 – UI viết cho team, không phải cá nhân

Một screen Flutter được viết gấp để kịp demo:

- widget dài
- nhiều UI ghép trực tiếp
- ít comment

Hãy phân tích:

- Dấu hiệu nào cho thấy screen này **chỉ người viết mới hiểu**?
- Khi dev khác vào sửa, rủi ro UI nào dễ xảy ra nhất?
- Với vai trò senior, bạn sẽ cảnh báo điều gì ngay từ đầu?

◈ Prompt 2 – Lạm dụng Scaffold làm mất tính nhất quán

Trong một app lớn:

- mỗi screen có Scaffold riêng
- AppBar, padding, background khác nhau

Hãy phân tích:

- Hậu quả về **UI consistency** khi app lớn dần
- Vì sao lỗi này **không lộ ở giai đoạn đầu**?
- Khi product yêu cầu đồng bộ UI, chi phí sửa nằm ở đâu?

◈ Prompt 3 – ListView.builder trong app dữ liệu lớn

Một ListView.builder hiển thị hàng trăm item:

- item có nhiều widget con
- onTap xử lý trực tiếp trong item

Hãy phân tích:

- Quyết định UI nào có thể gây **lag dù không crash**?
- Những dấu hiệu sớm cho thấy ListView đang trở thành bottleneck?
- Vì sao vấn đề này **thường chỉ xuất hiện khi app đã chạy thật**?

◈ Prompt 4 – Card UI bị copy–paste

Card được viết trực tiếp trong nhiều màn hình danh sách khác nhau.

Hãy phân tích:

- Khi design thay đổi, chi phí sửa UI nằm ở đâu?
- Vì sao copy–paste UI là "nợ kỹ thuật UI"?
- Reviewer nên phản ứng thế nào khi thấy pattern này?

◈ Prompt 5 – StatefulWidget phình to theo feature

Một screen ban đầu đơn giản, dần dần:

- thêm nhiều interaction
- thêm trạng thái UI

Tất cả vẫn nằm trong **1 StatefulWidget**.

Hãy phân tích:

- Dấu hiệu code UI đang "phình to" mất kiểm soát
- Khi thêm feature mới, rủi ro lớn nhất là gì?
- Vì sao lỗi này **không gây crash nhưng phá team**?

◈ Prompt 6 – setState rải rác trong screen lớn

Trong một screen:

- nhiều onPressed
- mỗi nơi đều gọi setState

Hãy phân tích:

PhucPT10@fpt.edu.vn

- Ảnh hưởng đến khả năng debug khi bug xuất hiện
- Vì sao càng nhiều setState, việc fix UI càng khó?
- Senior thường nhìn vào đâu để đánh giá rủi ro này?

◈ Prompt 7 – Callback không rõ trách nhiệm

onTap trong item ListView:

- vừa đổi UI
- vừa xử lý logic

Hãy phân tích:

- Vì sao điều này gây khó cho việc review code?
- Khi nhiều dev cùng làm, xung đột thường xảy ra ở đâu?
- Callback nên "nhẹ" đến mức nào ở Slot 6?

◈ Prompt 8 – UI danh sách không sẵn sàng mở rộng

Một screen danh sách:

- hiển thị ổn
- code gọn

Nhưng product sắp yêu cầu:

- chọn nhiều item
- trạng thái active
- nhiều hành động trên item

Hãy phân tích:

- UI hiện tại có dấu hiệu **chưa sẵn sàng cho mở rộng** ở đâu?
- Những quyết định Slot 6 nào ảnh hưởng trực tiếp đến việc mở rộng này?

◈ Prompt 9 – Widget Tree quá sâu trong dự án lớn

Widget Tree:

- lồng nhiều cấp
- khó hình dung cấu trúc

Hãy phân tích:

- Dấu hiệu nào cho thấy Widget Tree cần được refactor?
- Ảnh hưởng của tree sâu đến:
  - tốc độ phát triển
  - onboarding dev mới
- Vì sao tree sâu là "mùi code" trong UI?

◈ Prompt 10 – Debug UI trong môi trường team

UI bị lỗi layout:

- không crash
- không báo error

Hãy phân tích:

- Quy trình debug UI trong dự án thật khác gì so với bài tập cá nhân?
- Vai trò của reviewer trong việc phát hiện lỗi UI sớm
- Vì sao giao tiếp dev–reviewer quan trọng hơn code lúc này?

◈ Prompt 11 – UI thiếu nhất quán giữa các screen

Các screen đều dùng ListTile:

- nhưng padding, icon, text style khác nhau

Hãy phân tích:

- Ảnh hưởng đến trải nghiệm người dùng
- Vì sao lỗi này **tăng theo cấp số nhân** khi app lớn?
- Sửa sớm hay sửa muộn – cái nào đắt hơn? Vì sao?

PhucPT10@fpt.edu.vn

◈ Prompt 12 – UI chịu thay đổi yêu cầu liên tục

Product liên tục thay đổi:

- layout
- hành vi click
- trạng thái hiển thị

Hãy phân tích:

- UI Slot 6 cần được viết với tư duy gì để **không sợ thay đổi**?
- Quyết định UI nào khiến team phải "đập đi làm lại" nhiều lần?

◈ Prompt 13 – Review code junior: rủi ro tiềm ẩn

Một junior viết UI:

- code chạy
- không warning

Hãy phân tích:

- Những lỗi UI **không gây crash** nhưng nguy hiểm lâu dài
- Reviewer cần nhìn ra điều gì ngoài việc "chạy được"?

◈ Prompt 14 – Khi nào nên refactor UI sớm?

App đang chạy ổn:

- không bug lớn
- nhưng UI code khó đọc

Hãy phân tích:

- Dấu hiệu cho thấy nên refactor UI **trước khi quá muộn**
- Hậu quả nếu trì hoãn đến khi app lớn gấp đôi

◈ Prompt 15 – UI sẵn sàng cho scale chưa?

Đánh giá một screen Flutter theo góc nhìn:

- team 5–10 dev
- app phát triển 2–3 năm

Hãy phân tích:

- Screen này có đủ:
  - rõ ràng
  - nhất quán
  - dễ mở rộng không?
- Nếu chưa, vấn đề lớn nhất nằm ở **tư duy UI hay kỹ thuật UI**?

.

PhucPT10@fpt.edu.vn

PhucPT10@fpt.edu.vn